

Roll No: CS25M052
Collaborators (if any):

Name: Tushar

1. For dataset 1, you have to retrieve the first round key as well as the master key. We provide partial bytes of round-1 key for dataset 1 so you can validate your attack technique (see hint section).

Solution:

First round key(in hex): 3A 94 D6 1F 77 C2 8B 5E

Master key(in hex): 3A 94 D6 1F 77 C2 8B 5E A1 09

```
PS C:\Users\tusha\OneDrive\Desktop\CS25M052\present_data> python CS25M052.py present_plaintext_group1.npy present_traces_group1.npy present_ciphertext_group1.npy

===== FINAL RESULT =====
Round Key : 3A94D61F77C28B5E
Master Key: 3A94D61F77C28B5EA109
```

Figure 1: Screenshot output for Dataset-1

2. For dataset 2, no hint is provided and you are required to retrieve the first round key and master key.

Solution:

First round key(in hex): 01 23 45 67 89 AB CD EF

Master key(in hex): 01 23 45 67 89 AB CD EF 10 32

```
PS C:\Users\tusha\OneDrive\Desktop\CS25M052\present_data> python CS25M052.py present_plaintext_group2.npy present_traces_group2.npy present_ciphertext_group2.npy

===== FINAL RESULT =====
Round Key : 0123456789ABCDEF
Master Key: 0123456789ABCDEF1032
```

Figure 2: Screenshot output for Dataset-2

3. Your implementations are tested against hidden test cases also.

Solution: OK!!

4. Documentation of code/prompts and write-up mentioning the strategy used to find out the key.

Solution:

BACKGROUND ON PRESENT-80

PRESENT is a lightweight substitution–permutation network (SPN) block cipher with 64-bit block size and 80-bit master key. It consists of 31 rounds.

Each round contains:

1. AddRoundKey (XOR with 64-bit round key)
2. S-box layer (16 parallel 4-bit substitutions)
3. P-box layer (fixed bit permutation)

The first round key corresponds to the most significant 64 bits of the 80-bit key register. The lower 16 bits are not involved in the first round and therefore cannot be recovered directly through first-round side-channel leakage.

OVERALL STRATEGY FOR KEY RECOVERY

The objective of the attack was to recover the first round key and the full 80-bit master key of the PRESENT cipher using power traces and known plaintexts.

The final implementation consists of the following steps:

1. Identify a key-dependent intermediate value in the first round.
2. Model power consumption using the Hamming Weight (HW) leakage model.
3. Perform nibble-wise Correlation Power Analysis (CPA) to recover the first round key.
4. Store the top two key candidates per nibble.
5. Apply limited refinement (swap search up to depth 4) if necessary.
6. Recover the remaining 16 bits of the master key via brute-force and ciphertext validation.

Since PRESENT applies **AddRoundKey** followed by the S-box layer in the first round, the targeted intermediate value is:

$$v = S(P \oplus K)$$

where P is a plaintext nibble and K is the corresponding key nibble.

Because the PRESENT S-box operates on 4-bit inputs, the attack naturally targets individual nibbles of the state.

This intermediate depends only on one plaintext nibble and one key nibble, making it ideal for nibble-wise CPA.

LEAKAGE MODELING

Hamming Weight Model The final implementation uses the Hamming Weight leakage model:

$$L = HW(S(P \oplus K))$$

This model assumes that the instantaneous power consumption is proportional to the number of bits set to 1 in the processed value.

The Hamming Distance model was also considered conceptually, but experimental results showed that the Hamming Weight model produced significantly stronger correlation for this dataset.

CORRELATION POWER ANALYSIS IMPLEMENTATION

For each nibble position (0–15), the following steps were performed:

1. Iterate over all 16 possible key guesses.
2. For each guess, compute the hypothetical leakage vector across all traces.
3. Compute the Pearson correlation coefficient between the hypothetical leakage and each time sample of the measured traces:

$$\rho(k, t) = \frac{\sum_{i=1}^N (L_i(k) - \bar{L}(k))(T_{i,t} - \bar{T}_t)}{N \cdot \sigma_L \cdot \sigma_T}$$

4. For each key guess, record

$$\max_t |\rho(k, t)|$$

5. Rank the key guesses in descending order of correlation.

This process is repeated independently for all 16 nibbles.

TOP-2 CANDIDATE SELECTION AND UNCERTAINTY-BASED REFINEMENT

Instead of keeping only the highest-ranked key guess for each nibble, the implementation stores the top two candidates.

Let $k_j^{(1)}$ denote the rank-1 guess for nibble j , and $k_j^{(2)}$ denote the rank-2 guess.

The base round key is constructed as

$$K^{(0)} = (k_0^{(1)}, \dots, k_{15}^{(1)})$$

If this base key does not yield the correct master key, a limited refinement strategy is performed.

First, the difference between the rank-1 and rank-2 correlation scores is computed for each nibble. Nibbles with smaller score gaps are considered less certain and are prioritized for refinement.

The algorithm then performs a limited search where up to four uncertain nibbles may be replaced by their rank-2 candidates.

Formally, for swap depth $d \leq 4$,

$$K^{(d)} = \text{Replace } d \text{ positions of } K^{(0)} \text{ with } k_j^{(2)}$$

This uncertainty-based refinement is efficient because strong correlation values typically ensure that the correct key appears within the top two candidates.

MASTER KEY RECOVERY STRATEGY

PRESENT-80 uses an 80-bit master key. The first round key corresponds to the most significant 64 bits:

$$K_{\text{round1}} = \left\lfloor \frac{K_{\text{master}}}{2^{16}} \right\rfloor$$

Thus the remaining unknown portion consists of 16 bits. Candidate master keys are constructed as

$$K = (K_{\text{round1}} \ll 16) \mid x$$

where

$$x \in \{0, \dots, 2^{16} - 1\}$$

For each candidate key:

- Perform full PRESENT encryption.
- Validate against multiple plaintext-ciphertext pairs.

Using multiple validation pairs significantly reduces the probability of false positives. Since $2^{16} = 65,536$ the brute-force step is computationally feasible.

```
# --- UNCERTAINTY-BASED SEARCH ---
def find_master_key(all_ranked, test_pt, test_ct, max_swaps=4):
    """Prioritizes swapping nibbles with the smallest correlation gaps."""
    # Calculate confidence gap between Rank 1 and Rank 2
    gaps = [(r[0][1] - r[1][1], i) for i, r in enumerate(all_ranked)]
    gaps.sort() # Smallest gaps (most uncertain) first
    uncertain_nibs = [idx for _, idx in gaps]
    base_nibs = [r[0][0] for r in all_ranked]
    for n_swaps in range(max_swaps + 1):
        # Try every combination of n_swaps within the uncertainty list
        for to_swap in itertools.combinations(uncertain_nibs, n_swaps):
            current_nibs = list(base_nibs)
            for idx in to_swap:
                current_nibs[idx] = all_ranked[idx][1][0] # Try Rank-2 candidate

            rk = 0
            for i, n in enumerate(current_nibs):
                rk |= n << (4 * (15 - i))

            # Brute-force the remaining 16 bits of the 80-bit Master Key
            for last16 in range(65536):
                mk = (rk << 16) | last16
                # Verify with multiple PT/CT pairs to avoid false positives
                if all(present_encrypt(test_pt[i], mk) == test_ct[i] for i in range(len(test_pt))):
                    return rk, mk
    return None, None
```

Figure 3: Master key function

CODE STRUCTURE AND ORGANIZATION

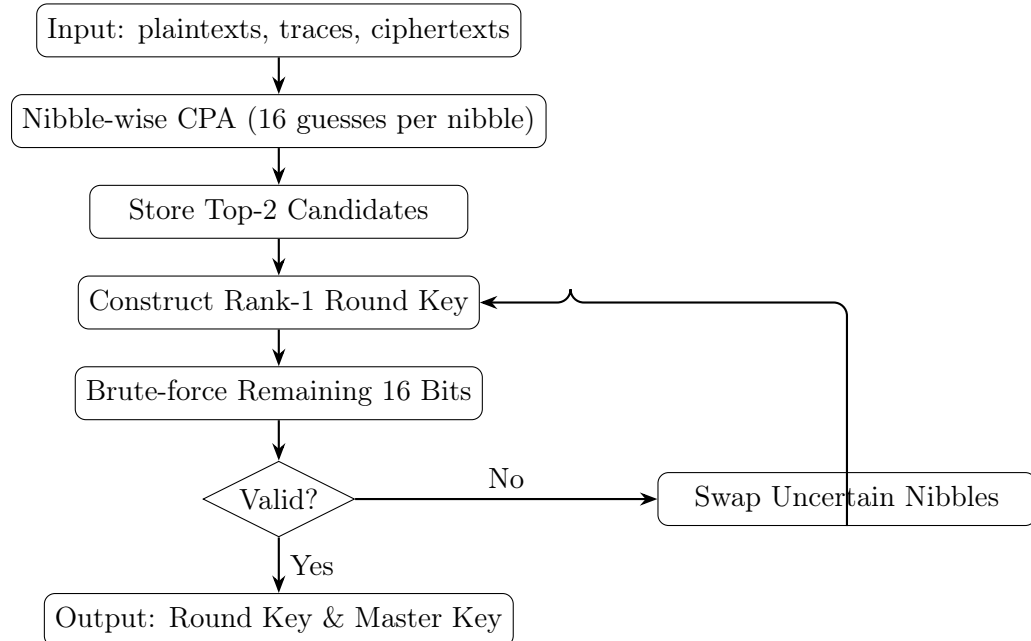


Figure 4: Flowchart of the Final Key Recovery Process

CODE DOCUMENTATION

Table 1: Function Summary

Function	Purpose
<code>sbox_layer(state)</code>	Applies the PRESENT S-box substitution to all 16 nibbles of the 64-bit state.
<code>pbox_layer(state)</code>	Performs the PRESENT bit permutation (P-box) on the 64-bit state.
<code>generate_round_keys(master_key)</code>	Generates the 32 round keys from the 80-bit master key according to the PRESENT key schedule.
<code>present_encrypt(plaintext, master_key)</code>	Performs full PRESENT encryption of a 64-bit plaintext using the generated round keys.
<code>recover_nibble_candidates(pt, traces)</code>	Performs nibble-wise Correlation Power Analysis (CPA). Computes correlation scores for all 16 key hypotheses for each nibble and returns ranked key candidates.
<code>find_master_key(all_ranked, test_pt, test_ct)</code>	Performs uncertainty-based nibble refinement and brute-forces the remaining 16 bits of the master key. Candidate keys are validated using known plaintext-ciphertext pairs.
<code>main()</code>	Loads the plaintext, trace, and ciphertext datasets, executes the CPA attack, performs key search, and outputs the recovered round key and master key.

LLM Usage Documentation

An LLM was used as a conceptual and debugging assistant during the development process. All attack design decisions, implementation details, and experimental validation were performed independently by the author. The LLM was consulted only for clarification of concepts, explanation

of algorithms, and improvement of documentation clarity. It was not used to generate the final attack solution.

LLM Used: ChatGPT 5.2

Prompt 1 — Understanding the PRESENT Cipher

Prompt: “Explain the internal working of the PRESENT-80 block cipher in detail. Describe the role of each component including AddRoundKey, the 4-bit S-box layer, the P-box permutation layer, and the key schedule.”

What I learned: The explanation clarified how the theoretical specification of the PRESENT cipher maps to an actual software implementation. In particular, it helped understand how the 64-bit state is divided into 16 nibbles for the S-box layer, how the P-box permutes individual bits of the state, and how the 80-bit master key is rotated and updated during the key schedule. This understanding helped ensure that the encryption implementation used for key verification was correct.

Prompt 2 — Clarifying Leakage Model Choice

Prompt: “In a first-round CPA attack on PRESENT, how can I decide whether Hamming Weight or Hamming Distance is the more appropriate leakage model?”

What I learned: The explanation highlighted that the Hamming Weight model is typically suitable when the device leaks the value written into a register, while the Hamming Distance model better represents transition-based leakage between two states. Empirical testing on the dataset showed that the Hamming Weight model produced significantly higher correlation values, confirming it as the appropriate leakage model for this implementation.

Prompt 3 — Vectorising Pearson Correlation

Prompt: “I have implemented Pearson correlation directly from the formula, but it is slow. How can I express the same computation using NumPy vector operations while preserving mathematical correctness?”

What I learned: The response explained that by centering the trace matrix once and computing a dot product between the centered leakage vector and the centered trace matrix, the Pearson correlation numerator can be evaluated for all time samples simultaneously. This vectorised formulation significantly improves performance while remaining mathematically equivalent to the original formula.

Prompt 4 — Handling Close Rank-1 and Rank-2 Candidates

Prompt: “If a few key candidates have very small differences between the highest and second-highest correlation scores, what is a principled way to verify correctness without introducing dataset-specific thresholds?”

What I learned: Instead of using arbitrary correlation thresholds, a refinement strategy can be applied by storing the top two candidates for each key position. Candidate combinations can then be verified using the cipher implementation as an oracle. This approach avoids dataset-specific tuning and ensures a robust and general solution.

Prompt 5 — Understanding Guessing Entropy in Side-Channel Attacks

Prompt: “In side-channel analysis, what does guessing entropy represent and how should guessing entropy vs number of traces plots be interpreted when comparing distinguishers such as Pearson correlation, Mutual Information, KL divergence, and Kolmogorov–Smirnov?”

What I learned: The explanation clarified that guessing entropy represents the average rank of the correct key hypothesis. A value of zero indicates that the correct key is ranked first. As the number of traces increases, effective distinguishers reduce the guessing entropy toward zero. This helped interpret the Task 5(c) plots, where Pearson correlation and Mutual Information converge quickly while other distinguishers may require more traces.

CONCLUSION

The attack successfully recovers both the 64-bit first-round key and the full 80-bit master key using only power traces and known plaintexts.

The combination of

- Hamming Weight leakage modeling
- Vectorized Pearson correlation
- Top-2 candidate storage
- Uncertainty-based nibble refinement
- Multi-pair validation

provides a robust and dataset-agnostic attack framework.

The results demonstrate that unprotected implementations of lightweight SPN ciphers such as PRESENT remain highly vulnerable to classical Correlation Power Analysis.

5. Answer the following questions:

- (a) Plot graphs showing how the correlation score of correct and wrong keys changes with the number of traces. The graph should have the correlation score on the y-axis and the number of traces on the x-axis. Refer to the PARAM paper for an example of what the graph should look like. (10 marks)
- (b) Perform CPA using the Hamming Weight approach as well as the Hamming Distance approach. Compare which of the two approaches is better, and why? (10 marks)
- (c) We have studied in class that Pearson's Correlation Coefficient used in CPA can be replaced with other statistical distance metrics, like, Mutual Information, KL Divergence, and Kolmogorov-Smirnov test. Experiment with these metrics on the data and compare the success rate (using the Guessing Entropy metric) across the various metrics. (10 marks)

Solution: a)

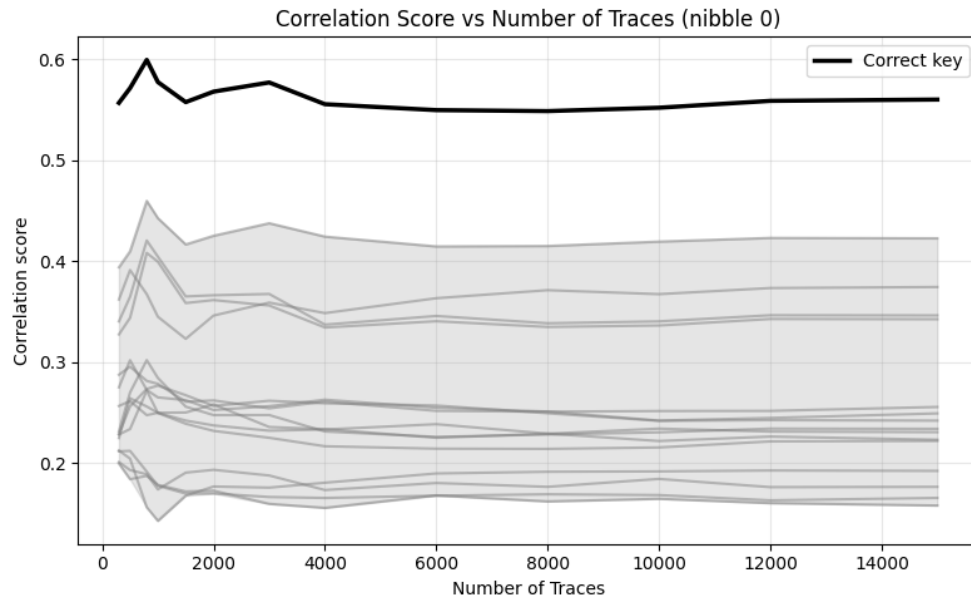


Figure 5: Correlation score vs. number of traces (Byte-0 Dataset-1)

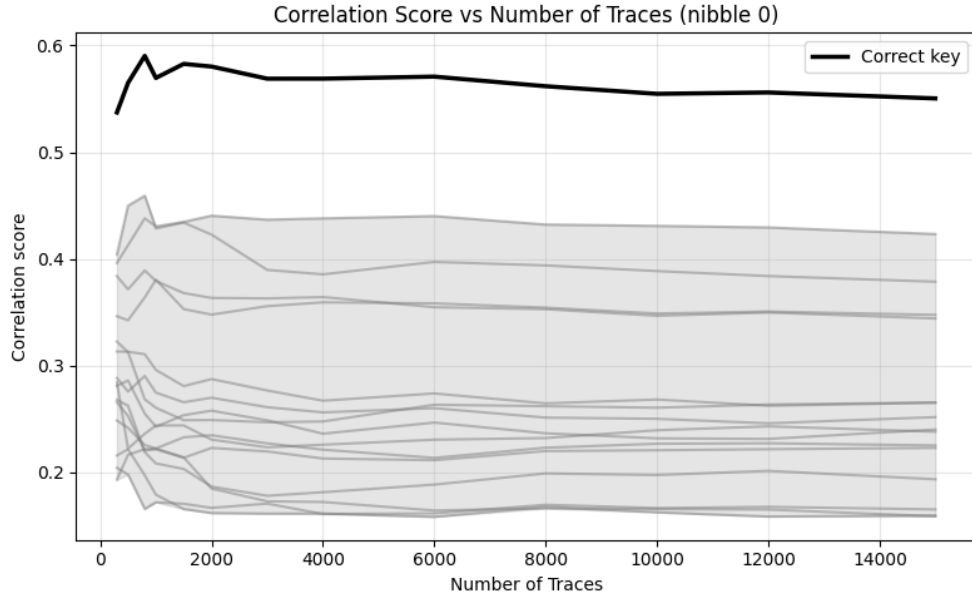


Figure 6: Correlation score vs. number of traces (Byte-0 Dataset-2)

The resulting plots show a clear separation between the correct key and incorrect key hypotheses. Even with a relatively small number of traces, the correct key produces a significantly higher correlation value compared to the other candidates. As the number of traces increases, the correlation of the correct key stabilizes and becomes consistently distinguishable from the remaining hypotheses.

This behavior reflects the statistical nature of CPA attacks. With more traces, random noise in the power measurements averages out, while the key-dependent leakage predicted by the Hamming Weight model remains consistent. Consequently, the correlation between the hypothetical leakage and the measured traces becomes stronger for the correct key and weaker for incorrect guesses.

The plots therefore demonstrate that only a moderate number of traces are sufficient to reliably identify the correct key hypothesis in the analyzed implementation of the PRESENT cipher.

b) To evaluate the suitability of different leakage assumptions, both the Hamming Weight (HW) and Hamming Distance (HD) models were tested on the same dataset.

The HW model assumes that power consumption is proportional to the number of bits set to 1 in the intermediate value:

$$L_{HW} = HW(S(P \oplus K)).$$

The HD model assumes that power consumption is proportional to the number of bit transitions between two states:

$$L_{HD} = HW(P \oplus S(P \oplus K)).$$

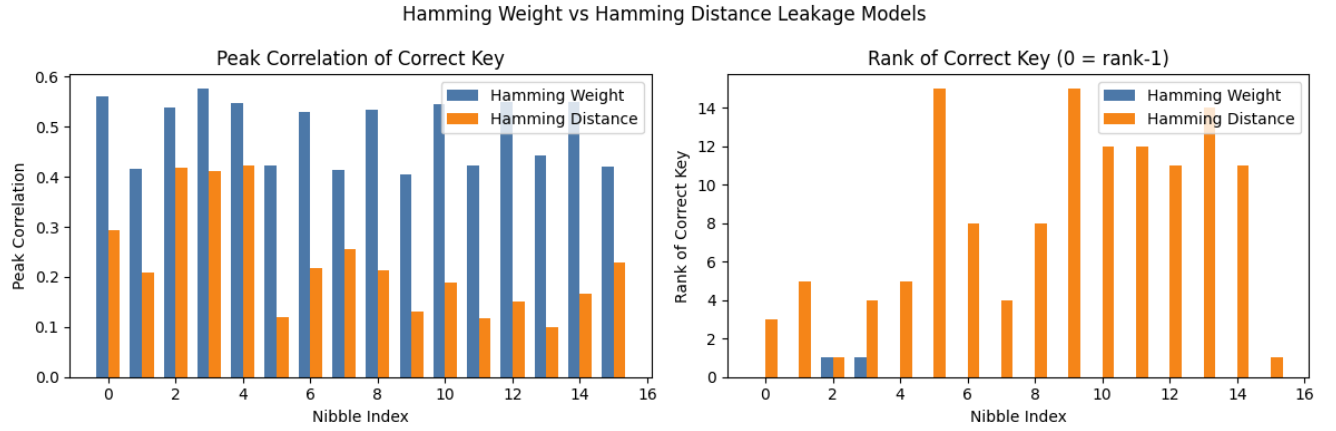


Figure 7: HW vs HD: peak correlation (left) and rank of correct key (right) for Dataset-1

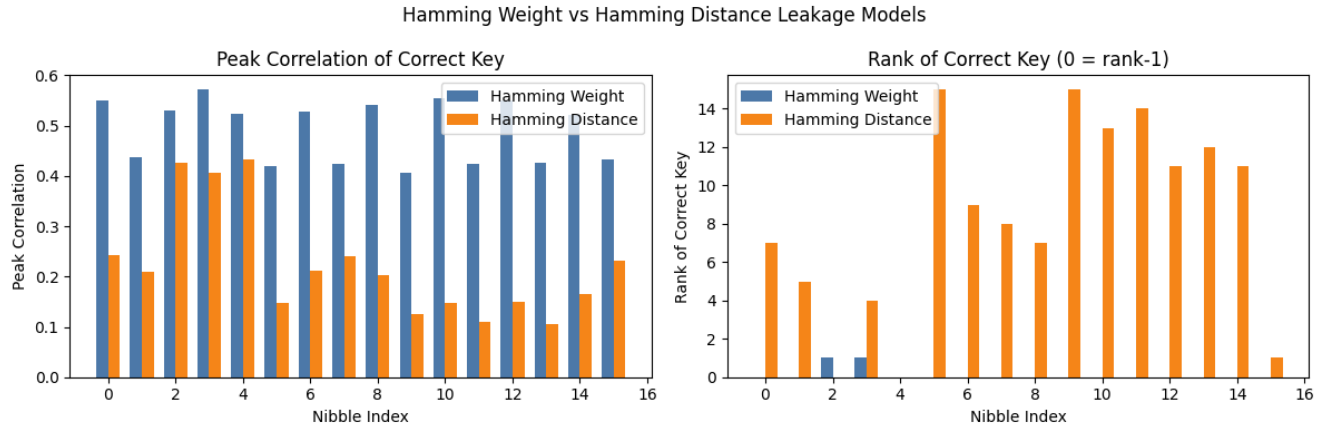


Figure 8: HW vs HD: peak correlation (left) and rank of correct key (right) for Dataset-2

To evaluate the effectiveness of different leakage assumptions, the attack was performed using two commonly used power leakage models:

- Hamming Weight (HW)
- Hamming Distance (HD)

For each nibble of the first round key, the maximum correlation value obtained using both models was computed. In addition, the rank of the correct key candidate was recorded for each nibble.

Peak Correlation Comparison

The left plots show the maximum absolute correlation obtained for the correct key hypothesis using both leakage models.

Across both datasets, the Hamming Weight model consistently produces significantly higher correlation values than the Hamming Distance model. The peak correlations for HW typically lie in the range of approximately 0.40–0.58, while HD correlations are substantially smaller, generally below 0.45 and often below 0.25.

This indicates that the HW model captures the dominant leakage behavior of the implementation more accurately than the HD model.

Key Rank Comparison

The right plots show the rank of the correct key candidate for each nibble.

For the Hamming Weight model, the correct key is recovered with rank 0 (best candidate) for almost all nibbles in both datasets. This confirms that the HW model provides a strong distinguisher for identifying the correct key.

In contrast, the Hamming Distance model performs poorly. The correct key often appears at significantly higher ranks, sometimes above rank 10. This means that HD fails to reliably distinguish the correct key from incorrect hypotheses.

Discussion

The superior performance of the Hamming Weight model suggests that the measured power leakage primarily reflects the number of bits set in the intermediate value rather than transitions between two states. This is consistent with many practical hardware implementations where power consumption is dominated by the value stored in registers rather than the switching activity between successive values.

Overall, the experimental results clearly demonstrate that the Hamming Weight model is significantly more effective than the Hamming Distance model for attacking this implementation of the PRESENT cipher.

c) Four different statistical distinguishers were evaluated in this experiment to measure how effectively the correct key hypothesis can be identified.

Pearson Correlation Pearson correlation is the classical distinguisher used in Correlation Power Analysis (CPA). It measures the linear relationship between the hypothetical leakage predicted by the leakage model and the measured power traces. The correlation coefficient is computed as

$$\rho = \frac{\sum_{i=1}^N (L_i - \bar{L})(T_i - \bar{T})}{N \cdot \sigma_L \cdot \sigma_T}$$

where L_i represents the predicted leakage values and T_i represents the measured trace samples. A higher absolute correlation indicates a stronger relationship between the predicted leakage and the measured power consumption. The correct key hypothesis is expected to produce the highest correlation.

Mutual Information (MI) Mutual Information measures the statistical dependence between two variables without assuming a linear relationship. In the context of side-channel analysis, MI measures how much information the observed power traces reveal about the predicted leakage values. Formally, mutual information is defined as

$$I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

where $p(x, y)$ is the joint probability distribution and $p(x)$ and $p(y)$ are the marginal distributions. A larger mutual information value indicates stronger dependence between the key hypothesis and the observed traces.

KL Divergence Kullback–Leibler (KL) divergence measures the difference between two probability distributions. In side-channel attacks, it is often used to compare the distribution of traces corresponding to different predicted leakage classes. The KL divergence between two distributions P and Q is defined as

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

A larger divergence indicates that the distributions are more distinguishable, which helps identify the correct key hypothesis.

Kolmogorov–Smirnov (KS) Statistic The Kolmogorov–Smirnov statistic measures the maximum distance between the empirical cumulative distribution functions (CDFs) of two trace distributions. In this attack, traces are partitioned according to the predicted leakage classes, and the KS statistic is computed as

$$D = \sup_x |F_1(x) - F_2(x)|$$

where $F_1(x)$ and $F_2(x)$ are the cumulative distribution functions of the two trace groups. A larger KS value indicates that the distributions differ significantly, which can reveal the correct key candidate.

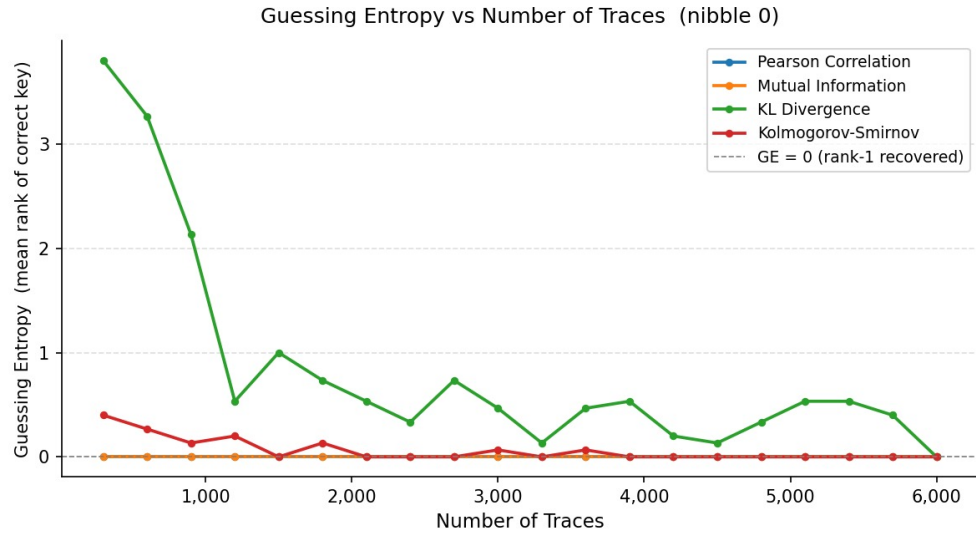


Figure 9: Dataset 1 — Guessing Entropy vs Number of Traces (nibble 0)

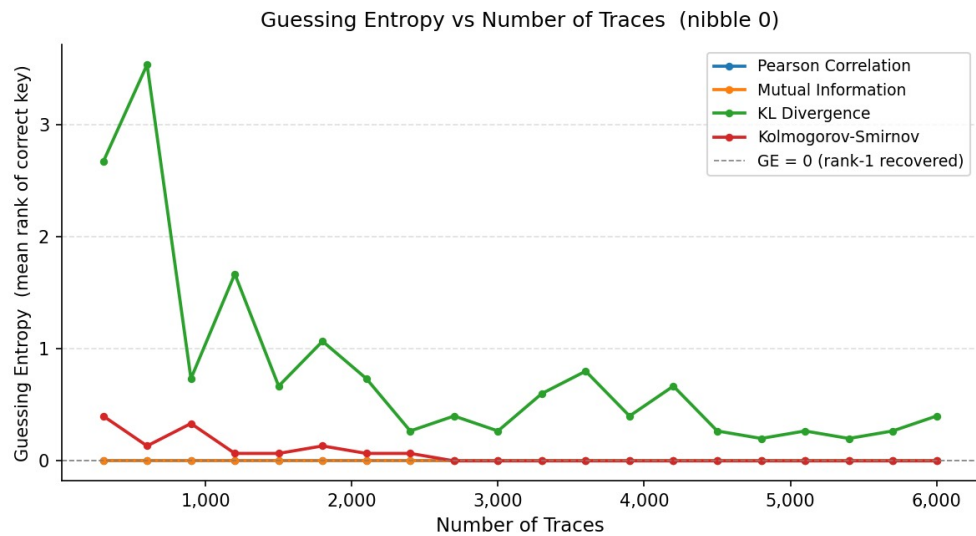


Figure 10: Dataset 2 — Guessing Entropy vs Number of Traces (nibble 0)

Guessing Entropy vs Number of Traces

Figures 9 and 10 show the evolution of guessing entropy as the number of traces increases for nibble 0. As the number of traces increases, the attack gains more statistical information about the secret key, allowing the correct key candidate to move toward rank 1.

The results demonstrate that **Pearson Correlation** and **Mutual Information** achieve $GE = 0$ very quickly and remain stable across all trace counts. This indicates that these two metrics are highly effective in capturing the linear and statistical dependency between the leakage model and the measured traces.

The **Kolmogorov–Smirnov (KS)** statistic also performs well, although it initially exhibits small fluctuations before stabilizing at $GE = 0$. This behavior is expected because KS measures distribution differences rather than direct correlation, which can require slightly more data to converge.

In contrast, the **KL Divergence** metric shows significantly higher guessing entropy values during the early stages of the attack. Although the GE gradually decreases as the number of traces increases, it fluctuates considerably and only approaches low values at higher trace counts. This suggests that KL divergence is less stable and less reliable for this particular leakage model.

Overall, Pearson correlation and mutual information demonstrate the most consistent and reliable key recovery performance.

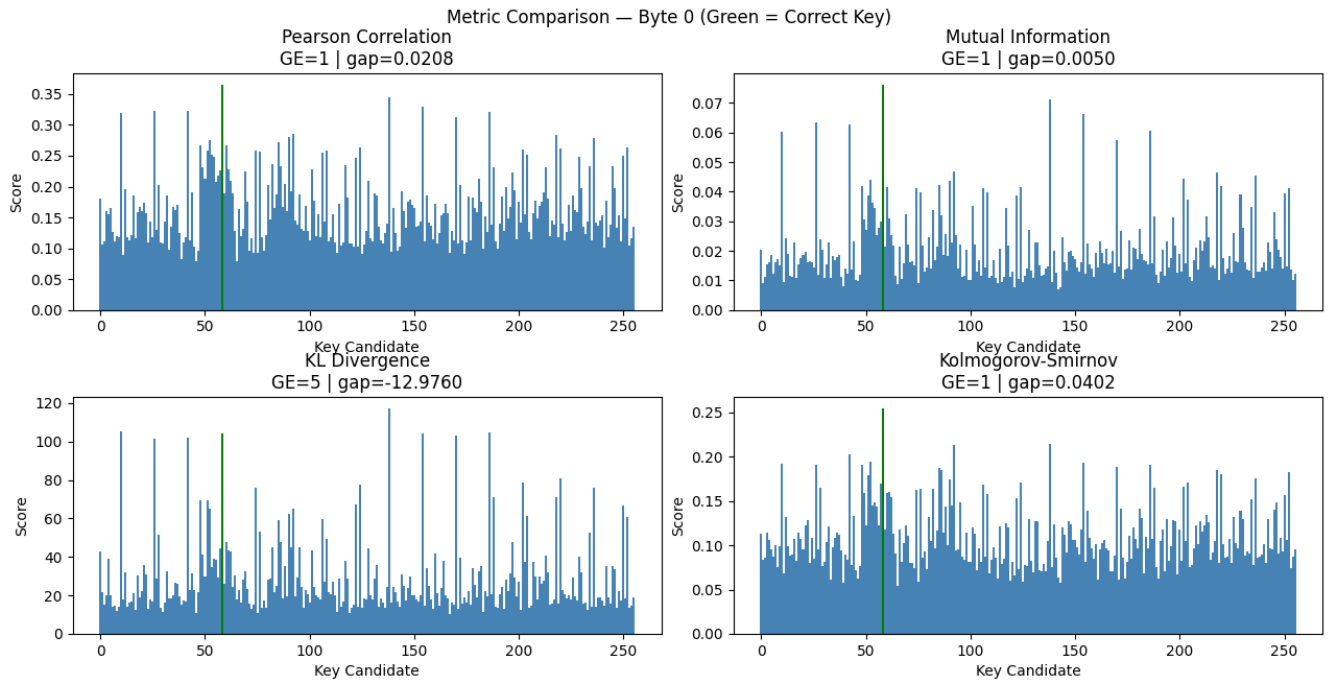


Figure 11: Comparison of different statistical metrics (Pearson correlation, Mutual Information, KL Divergence, and Kolmogorov–Smirnov statistic) across all 256 key candidates for Byte 0. The correct key candidate is highlighted in green.

Metric Comparison for Key Candidates

Figure 11 presents the score distribution across all 256 key candidates for the evaluated metrics. The correct key is highlighted in green.

For **Pearson correlation**, the correct key produces the highest score, leading to a guessing entropy of 1. The clear separation between the correct key and other candidates indicates strong correlation between the predicted leakage and the observed traces.

The **Mutual Information** metric also successfully identifies the correct key with a similar GE value, although the score differences between candidates are slightly smaller.

The **Kolmogorov–Smirnov statistic** shows a clearer separation compared to many other candidates, indicating good sensitivity to distribution differences in leakage.

However, **KL divergence** performs poorly in this comparison. The correct key does not produce the highest score, resulting in a larger guessing entropy ($GE = 5$). Additionally, the score gap between the correct key and other candidates is negative, indicating that several incorrect key guesses appear statistically stronger than the correct one.

Discussion From the experimental results, it can be observed that correlation-based and information-theoretic metrics behave differently depending on the leakage characteristics.

- **Pearson Correlation** performs consistently well because it directly measures linear dependency between predicted leakage and observed traces.
- **Mutual Information** is also highly effective as it captures both linear and non-linear dependencies.
- **Kolmogorov–Smirnov** provides moderate performance by comparing empirical distributions.
- **KL Divergence** is sensitive to distribution estimation errors and therefore performs poorly with limited trace data.

Overall, the results suggest that **Pearson correlation and mutual information are the most reliable metrics for key recovery in this experimental setup.**